

GymControl - Documentación técnica

Portada



Materia: Algoritmos y Estructura de Datos 1 / Programación 1

Proyecto: Sistema de gestión para gimnasios

Comisión: lunes por la noche - N.º 577502

Integrantes: Rodrigo García Solá, Lucas Cragaris, Tomás van Nynatten, Javier Candalaft y Edney Ribeiro

Repositorio: <https://github.com/rodrings/GymControl>

Fecha: 3 de septiembre de 2026

El docente confirmó que podemos presentar el trabajo entre los cinco integrantes.

1. Objetivo y problemática

GymControl reemplaza registros manuales dispersos por un programa de consola que centraliza socios, clases e inscripciones. Su objetivo es mantener relaciones consistentes, controlar cupos y facilitar búsquedas, ordenamientos y estadísticas básicas.

La aplicación trabaja únicamente en memoria: no utiliza archivos de datos, base de datos, interfaz gráfica ni programación orientada a objetos.

2. Cómo ejecutar

Se requiere Python 3.9 o posterior.

```
python3 tp.py
```

Credenciales de demostración:

- Usuario: admin
- Contraseña: admin1234

El login admite hasta tres intentos. Al salir, todas las modificaciones hechas durante la sesión se pierden.

3. Estructura general

El archivo tp.py está organizado en estos bloques:

1. Constantes de índices y matrices con datos iniciales.
2. Funciones estadísticas basadas en matrices.
3. Búsquedas y ordenamientos.
4. Validaciones y funciones auxiliares.
5. Gestión de socios.
6. Gestión de clases.
7. Gestión de inscripciones.
8. Menús y programa principal.

Flujo principal:

```
login()
-> main_menu()
    -> affiliate_menu()
    -> clases_menu()
    -> inscription_menu()
    -> opcion_menu_ordenamiento()
    -> opcion_menu_busqueda()
    -> matrix_menu()
```

4. Matrices y listas de cadenas

Cada entidad se representa mediante una lista de listas. Las constantes evitan números mágicos al acceder a las columnas.

Matriz	Formato de fila	Contenido textual
login_users	[usuario, contraseña]	Usuario y contraseña.

Matriz	Formato de fila	Contenido textual
affiliates	[nombre, código, edad, tipo, teléfono]	Nombre y teléfono.
classes	[código, nombre, nivel, cupos]	Nombre de la clase.
enrollments	[código, socio, clase, asistencias, estado]	Relaciona códigos de otras matrices.

Valores codificados:

- Tipo de socio: 1 = Mensual, 2 = Libre, 3 = Premium.
- Nivel de clase: 1 = Principiante, 2 = Intermedio, 3 = Avanzado.
- Estado de inscripción: 1 = Activa, 2 = Inactiva.

La matriz de socios posee una columna de teléfono. La PR #8 incorporó su validación mediante regex y completó su uso en el alta, la modificación, el listado y la confirmación de baja.

5. Funciones estadísticas

Función	Descripción
affiliates_by_class_matrix()	Construye una matriz donde cada fila agrupa códigos de socios por clase.
affiliates_by_class()	Muestra la cantidad de inscripciones asociadas a cada clase.
attendances_by_class_matrix()	Agrupar asistencias según la clase correspondiente.
total_attendances_by_class()	Usa map y reduce para calcular el total de asistencias de cada clase.
enrollment_by_class_level_matrix()	Agrupar inscripciones en tres filas, una por nivel.
affiliates_by_type_and_class_matrix()	Construye una matriz de tres tipos de socio por cantidad de clases.

6. Búsquedas y ordenamientos

Función	Técnica	Resultado
search_position()	filter y una condición lambda recibida como parámetro.	Primera posición que cumple la condición o -1.
buscar_clase_binaria()	Ordena copias de las columnas por código y aplica búsqueda binaria.	Clase encontrada o mensaje de ausencia.
buscar_socio_secuencial()	Búsqueda secuencial por código.	Posición y datos del socio.
ordenar_socios_por_edad()	Selección.	Copia de socios ordenada por edad.
ordenar_clases_por_nivel()	Inserción.	Copia de clases ordenada por nivel.
ordenar_inscripciones_por_asistencias()	Burbujeo.	Copia de inscripciones ordenada por asistencias.

Los ordenamientos trabajan sobre copias de filas y no modifican las matrices originales.

7. Validaciones y funciones auxiliares

Función	Descripción
es_entero()	Comprueba si un valor puede convertirse a entero.

Función	Descripción
<code>es_flotante()</code>	Comprueba si un valor puede convertirse a decimal.
<code>es_string()</code>	Comprueba si un valor puede convertirse a cadena.
<code>es_nombre_clase_valido()</code>	Regex que acepta letras, espacios, tildes y ñ.
<code>pedir_entero()</code>	Repite una entrada hasta obtener un entero y, opcionalmente, respetar límites.
<code>get_level_name()</code>	Traduce el código de nivel a texto.
<code>get_type_name()</code>	Traduce el código de tipo de socio a texto.
<code>search_position()</code>	Usa <code>filter</code> y una condición para obtener la posición de la primera fila coincidente.
<code>search_class_position()</code>	Reutiliza <code>search_position()</code> con una lambda sobre el código de clase.
<code>search_affiliate_position()</code>	Reutiliza <code>search_position()</code> con una lambda sobre el código de socio.
<code>search_inscription_position()</code>	Devuelve el índice de una inscripción o -1.
<code>search_user_position()</code>	Busca el usuario mediante <code>re.fullmatch</code> sin distinguir mayúsculas.
<code>does_class_code_exist()</code>	Informa si existe un código de clase.
<code>does_affiliate_code_exist()</code>	Informa si existe un código de socio.
<code>remove_enrollment_at()</code>	Elimina una fila de inscripción por posición.
<code>remove_enrollments_by_affiliate()</code>	Elimina las inscripciones asociadas a un socio.
<code>remove_enrollments_by_class()</code>	Elimina las inscripciones asociadas a una clase.
<code>input_option()</code>	Valida una opción entre 0 y 4; quedó definida pero no se usa en el flujo actual.

8. Login

Función	Descripción
<code>login()</code>	Solicita usuario y contraseña, reutiliza <code>search_user_position()</code> y bloquea después de tres intentos fallidos.

`search_user_position()` usa la bandera `re.IGNORECASE`, por lo que `ADMIN` y `admin` encuentran el mismo usuario. La contraseña sí distingue mayúsculas y minúsculas.

9. Gestión de socios

Función	Descripción
<code>sumar_afiliados()</code>	Solicita nombre, edad, tipo y teléfono validado; genera el próximo código y agrega una fila.
<code>eliminar_afiliados()</code>	Solicita código y confirmación, elimina el socio y sus inscripciones asociadas.
<code>modify_affiliate()</code>	Permite conservar o reemplazar nombre, edad, tipo y teléfono.
<code>list_affiliates()</code>	Muestra código, nombre, edad, tipo y teléfono.

`es_telefono_valido()` exige el formato `xx-xxxx-xxxx` antes de guardar un teléfono nuevo o modificado.

10. Gestión de clases

Función	Descripción
<code>sumar_clase()</code>	Valida nombre, nivel y capacidad; genera el código y agrega la fila.
<code>eliminar_clase()</code>	Elimina una clase y sus inscripciones después de confirmar.

Función	Descripción
<code>modify_clase()</code>	Permite cambiar nombre, nivel y capacidad.
<code>list_clases()</code>	Muestra una tabla con código, nombre, nivel y cupos disponibles.

11. Gestión de inscripciones

Función	Descripción
<code>is_affiliate_enrolled_in_class()</code>	Usa <code>filter</code> y una <code>lambda</code> para detectar una inscripción activa equivalente.
<code>alta_inscripcion()</code>	Valida clase, cupo, socio y duplicados; crea una inscripción activa y descuenta un cupo.
<code>list_inscripciones()</code>	Resuelve los códigos y muestra nombres, asistencias y estado.
<code>clases_de_socio()</code>	Usa <code>filter</code> para elegir inscripciones activas y <code>map</code> para convertirlas en nombres de clase.
<code>listar_clases_socio()</code>	Solicita un socio y presenta sus clases activas.
<code>baja_inscripcion()</code>	Cambia una inscripción activa a inactiva y devuelve un cupo.
<code>modify_inscripcion()</code>	Modifica asistencias o estado y ajusta cupos al activar o desactivar.

12. Menús

Función	Responsabilidad
<code>input_clases_option()</code>	Valida opciones del menú de clases.
<code>clases_menu()</code>	Ejecuta el ABM de clases.
<code>input_affiliate_option()</code>	Valida opciones del menú de socios.
<code>affiliate_menu()</code>	Ejecuta el ABM de socios.
<code>input_inscription_option()</code>	Valida opciones del menú de inscripciones.
<code>inscription_menu()</code>	Ejecuta el ABM de inscripciones y el listado por socio.
<code>menu_busqueda()</code>	Valida opciones de búsqueda.
<code>opcion_menu_busqueda()</code>	Ejecuta búsquedas hasta volver al menú principal.
<code>menu_ordenamiento()</code>	Valida opciones de ordenamiento.
<code>opcion_menu_ordenamiento()</code>	Ejecuta el ordenamiento seleccionado.
<code>input_matrix_option()</code>	Valida opciones de estadísticas.
<code>matrix_menu()</code>	Ejecuta los reportes matriciales.
<code>input_main_option()</code>	Valida opciones de 0 a 6.
<code>main_menu()</code>	Coordina todos los submenús.

13. Uso de `map`, `filter`, `reduce` y `regex`

`map` y `reduce`

`total_attendances_by_class()` aplica `map` sobre las listas de asistencias de cada clase. Dentro de cada elemento utiliza `reduce` con acumulador inicial 0 para obtener el total.

`filter`

`search_position()` filtra índices aplicando una condición lambda sobre cada fila.
`is_affiliate_enrolled_in_class()` filtra inscripciones por socio, clase y estado activo.
`clases_de_socio()` filtra las inscripciones activas del socio antes de mapearlas a nombres.

Expresiones regulares

`es_nombre_clase_valido()` valida nombres mediante el patrón `^[A-Za-zÁÉÍÓÚáéíóúÑñ\s]+$.
search_user_position() usa re.fullmatch(..., re.IGNORECASE) para comparar usuarios.
es_telefono_valido() valida teléfonos con el patrón \d{2}-\d{4}-\d{4}.`

14. Capturas de funcionamiento

Inicio de sesión y menú principal

```
python tp.py
ingrese su nombre de usuario: admin
ingrese su contraseña: admin1234
¡Bienvenido al sistema de gestión del gimnasio!
--- MENU PRINCIPAL ---
Opción 1: Gestión de afiliados
Opción 2: Gestión de clases
Opción 3: Gestión de inscripciones
Opción 4: Ordenamiento
Opción 5: Búsqueda
Opción 6: Cálculos estadísticos matriciales
Opción 0: Salir
Ingrese una opción: █
```

Consulta de socios y clases

```
Ingrese una opción: 5
--- MENÚ DE BÚSQUEDA ---
Opción 1: Buscar clase por código (Binaria)
Opción 2: Buscar socio por código (Secuencial)
Opción 0: Volver al menú principal
Ingrese una opción: 2
BÚSQUEDA SECUENCIAL DE SOCIO POR CÓDIGO
Ingrese el código del socio a buscar: 101
Socio encontrado en posición 0:
Código: 101, Nombre: Juan Perez, Edad: 28, Tipo: Mensual
--- MENÚ DE BÚSQUEDA ---
Opción 1: Buscar clase por código (Binaria)
Opción 2: Buscar socio por código (Secuencial)
Opción 0: Volver al menú principal
Ingrese una opción: █
```

Reportes matriciales

```
--- MENU PRINCIPAL ---
Opción 1: Gestión de afiliados
Opción 2: Gestión de clases
Opción 3: Gestión de inscripciones
Opción 4: Ordenamiento
Opción 5: Búsqueda
Opción 6: Cálculos estadísticos matriciales
Opción 0: Salir
Ingrese una opción: 6
--- MENÚ DE CÁLCULOS ESTADÍSTICOS MATRICIALES ---
Opción 1: Matriz de cantidad de afiliados por clase
Opción 2: Matriz de cantidad de inscripciones por nivel de clase
Opción 3: Total de asistencias por clase
Opción 4: Matriz de cantidad de afiliados por tipo y clase
Opción 0: Volver al menú principal
Ingrese una opción: 3
TOTAL DE ASISTENCIAS POR CLASE
Clase: Yoga - Total de asistencias: 28
Clase: Crossfit - Total de asistencias: 0
Clase: Boxeo - Total de asistencias: 30
Clase: Pilates - Total de asistencias: 8
Clase: Musculacion - Total de asistencias: 16
Clase: Spinning - Total de asistencias: 0
Clase: Funcional - Total de asistencias: 0
Clase: Zumba - Total de asistencias: 0
Clase: Natacion - Total de asistencias: 0
Clase: Stretching - Total de asistencias: 0
--- MENÚ DE CÁLCULOS ESTADÍSTICOS MATRICIALES ---
Opción 1: Matriz de cantidad de afiliados por clase
Opción 2: Matriz de cantidad de inscripciones por nivel de clase
Opción 3: Total de asistencias por clase
Opción 4: Matriz de cantidad de afiliados por tipo y clase
Opción 0: Volver al menú principal
Ingrese una opción: █
```

Las capturas se generaron a partir de ejecuciones reales del tp.py correspondiente a esta rama.

15. Aportes y ramas por integrante

Integrante	Rama	Pull requests y aporte
Javier Candalaft	dev-canda	Base del sistema, migración desde JSON y PR #5: matrices de usuarios, reduce, login, regex, documentación y PDFs.
Lucas Cragaris	dev-lucas	PR #1: documentación y clases activas por socio. PR #7: prevención de duplicados y regex con espacios/tildes.
Rodrigo García Solá	dev-rodri	PR #3: refactor completo de las estructuras a matrices e integración de sus operaciones.
Tomás van Nynatten	dev-tom	PR #4: teléfonos precargados. PR #8 integrada: validación y CRUD del teléfono.

Integrante	Rama	Pull requests y aporte
Edney Ribeiro	dev-ed	PR #9: función genérica <code>search_position()</code> y reutilización de <code>filter</code> y lambdas en búsquedas de socios y clases.

16. Alcance y decisiones

- La integración incluye la gestión completa de teléfonos de la PR #8 y la búsqueda genérica de la PR #9.
- El docente confirmó que podemos presentar el trabajo entre los cinco.
- La información se administra en memoria y se reinicia al cerrar el programa, de acuerdo con el alcance definido.
- Los casos límite de liberación de cupos al eliminar socios y de referencias manualmente alteradas no forman parte de los requisitos funcionales obligatorios.
- Los flujos principales de socios, clases, inscripciones, búsquedas, ordenamientos y reportes fueron probados.

17. Trazabilidad final

Elemento exigido	Ubicación
Portada, objetivo, alcance y problemática	Portada y secciones 1 a 3 de este documento; propuesta en <code>GymControl.md</code>
Funcionalidades y estructura	Secciones 3 a 12 de este documento
Matrices y listas de strings	Sección 4
<code>map</code> , <code>filter</code> , <code>reduce</code> y <code>regex</code>	Sección 13
Capturas	Sección 14 y <code>docs/capturas/</code>
Repositorio	Portada y README
Historial y aporte individual	Sección 15 y <code>Modificaciones.md</code>
Pendientes	Sección 16 y <code>Modificaciones.md</code>

Los contenidos técnicos, la documentación y la evidencia individual en Git están cubiertos para la entrega escrita. La comisión está completa y el docente confirmó que podemos presentarlo entre los cinco.